Prueba escrita Temas 3 y 4 — Pila, Cola y Árboles

Solución de referencia

MODELO A — Ejercicio 1 (2 p.) `ColaEnlazada::desencolar`

```
bool ColaEnlazada::desencolar() {
    if (primero_ == nullptr) return false;
    Nodo* aBorrar = primero_;
    primero_ = primero_->siguiente;
    if (primero_ == nullptr) ultimo_ = nullptr;   // se ha quedado vacia
    delete aBorrar;
    n_--;
    return true;
}
```

MODELO A — Ejercicio 2 (2 p.) Traza de montículo de máximos

(a) Arrays tras cada inserción: `[15]`, `[40 15]`, `[40 15 25]`, `[50 40 25 15]` (50 sube dos veces), `[50 40 25 15 10]`, `[50 40 45 15 10 25]` (45 sube una vez). **Final:** `50 40 45 15 10 25`.

(b) El último (25) pasa a la raíz: `[25 40 45 15 10]`; baja por el **mayor** de sus hijos (45). **Resultado:** `45 40 25 15 10`.

(c) No: el 30 (índice 1) es menor que su hijo 35 (índice $2 \cdot 1 + 1 = 3$).

MODELO A — Ejercicio 3 (2,5 p.) `inordenIterativo` con pila

```
std::vector<int> inordenIterativo(Nodo* raiz) {
    std::vector<int> salida;
    std::stack<Nodo*> pila;
    Nodo* actual = raiz;
    while (actual != nullptr || !pila.empty()) {
        while (actual != nullptr) {
            pila.push(actual);
            actual = actual->izq;
        }
        actual = pila.top();
        pila.pop();
        salida.push_back(actual->valor);
        actual = actual->der;
    }
    return salida;
}
```

```
std::vector<int> inordenIterativo(Nodo* raiz) {
    std::vector<int> salida;
    std::stack<Nodo*> pila;
    Nodo* actual = raiz;
    while (actual != nullptr || !pila.empty()) {
        if (actual != nullptr) {
            pila.push(actual);
            actual = actual->izq;
        } else {
            actual = pila.top();
            pila.pop();
            salida.push_back(actual->valor);
            actual = actual->der; }
    }
    return salida;
}
```

MODELO A — Ejercicio 4 (3,5 p.) Organización compartimentada

(a) `reclutar` (1,5 p.). El nuevo **no** puede ser el enlace si ya lo hay: el jefe cambiaría de contacto y habría que repuntar el `contacto` de toda la célula ($O(k)$). Se coloca justo detrás del enlace.

```
Miembro* Organizacion::reclutar(Miembro* jefe, const std::string& alias) {
    Miembro* nuevo;
    if (jefe->enlace == nullptr) {              // primer subordinado: sera el enlace
        nuevo = new Miembro(alias, jefe);
        jefe->enlace = nuevo;
    } else {
        Miembro* e = jefe->enlace;
        nuevo = new Miembro(alias, e);          // informa al ENLACE, no al jefe
        nuevo->companero = e->companero;        // primero engancha el resto...
        e->companero = nuevo;                   // ...luego se coloca tras el enlace
    }
    n_++;
    return nuevo;
}
```

(b) `difundirPorRangos` (2 p.). Es el recorrido por niveles del tema 4 sobre primer-hijo/hermano-derecho: los hijos de `x` son la cadena `x->enlace`, `->companero`, ...

```
void Organizacion::difundirPorRangos(Miembro* origen, const std::string& msg) {
    std::queue<Miembro*> pendientes;
    pendientes.push(origen);
    while (!pendientes.empty()) {
        Miembro* actual = pendientes.front();
        pendientes.pop();
        for (Miembro* m = actual->enlace; m != nullptr; m = m->companero) {
            entregar(m, msg);
            pendientes.push(m);
        }
    }
}
```

MODELO B — Ejercicio 1 (2 p.) `ColaCircular::encolar`

```
void ColaCircular::encolar(int valor) {
    if (n_ == capacidad_) redimensionar(capacidad_ * 2);   // tambien vale *1.5
    datos_[(frente_ + n_) % capacidad_] = valor;            // DESPUES de redimensionar
    n_++;
}
```

MODELO B — Ejercicio 2 (2 p.) Traza de árbol binario de búsqueda

Árbol tras insertar: raíz 50; izquierdo 30 (con 20 y 40; 40 con 35 y 45); derecho 70 (con 60 y 80).

(a) Postorden: `20 35 45 40 30 60 80 70 50.` Altura: **3** (camino 50–30–40–35).

(b) El 30 tiene dos hijos: su sucesor es el mínimo del subárbol derecho, 35 (hoja). Se copia 35 en el nodo y se borra la hoja. Preorden: `50 35 20 40 45 70 60 80.`

(c) El 50 tiene dos hijos: su sucesor es 60 (hoja). Preorden: `60 35 20 40 45 70 80.`

MODELO B — Ejercicio 3 (2,5 p.) `anchuraMaxima` con cola

```
int anchuraMaxima(Nodo* raiz) {
    if (raiz == nullptr) return 0;
    std::queue<Nodo*> pendientes;
    pendientes.push(raiz);
    int maxima = 0;
    while (!pendientes.empty()) {
        int enEsteNivel = pendientes.size();      // FOTO del nivel antes de tocarlo
        maxima = std::max(maxima, enEsteNivel);
        for (int i = 0; i < enEsteNivel; i++) {
            Nodo* actual = pendientes.front();
            pendientes.pop();
            if (actual->izq != nullptr) pendientes.push(actual->izq);
            if (actual->der != nullptr) pendientes.push(actual->der);
        }
    }
    return maxima;
}
```

MODELO B — Ejercicio 4 (3,5 p.) Organización compartimentada

(a) `jefeDirecto` e `informar` (1,5 p.). El contacto de un miembro es su jefe sólo si es el enlace; si no, su contacto es el enlace de su célula, cuyo contacto sí es el jefe.

```
Miembro* Organizacion::jefeDirecto(Miembro* m) const {
    if (m->contacto == nullptr) return nullptr;          // el capo
    if (m->contacto->enlace == m) return m->contacto;    // m es el enlace
    return m->contacto->contacto;                        // contacto = enlace de su celula
}

int Organizacion::informar(Miembro* m, const std::string& msg) {
    int saltos = 0;
    for (Miembro* p = m->contacto; p != nullptr; p = p->contacto) {
        entregar(p, msg);
        saltos++;
    }
    return saltos;
}
```

(b) `difundir` recursivo (2 p.). Preorden sobre primer-hijo/hermano-derecho empezando por la célula subordinada de `origen`.

```
void Organizacion::difundir(Miembro* origen, const std::string& msg) {
    for (Miembro* m = origen->enlace; m != nullptr; m = m->companero) {
        entregar(m, msg);
        difundir(m, msg);        // antes de pasar al siguiente: en profundidad
    }
}
// Variante equivalente, con doble recursion:
//   void aux(Miembro* x, msg) { if (!x) return; entregar(x, msg);
//                               aux(x->enlace, msg); aux(x->companero, msg); }
//   difundir(origen, msg) { aux(origen->enlace, msg); }
```